PRML Homework 1

Zi Lin
Email:linzi_15762331896@163.com
March 20, 2026

Abstract

This work evaluates multiple regression methods on a dataset where the relationship between input and output is clearly nonlinear. Three optimization algorithms—least squares, gradient descent, and Newton's method—are first applied to fit a linear model. The results show consistently high prediction errors and low R^2 scores, indicating that a straight line cannot adequately represent the data.

To overcome this limitation, we examine polynomial regression with degrees ranging from 1 to 20. The model achieves its strongest test performance at degree 11, after which further increasing the degree leads to noticeable overfitting. As an alternative, we implement kernel regression using a Gaussian kernel, which delivers superior generalization when the bandwidth is set to 0.15, outperforming both linear and polynomial models.

The experiments reveal that selecting an appropriate model structure is critical for this dataset. Among the tested approaches, kernel regression proves to be the most effective, though its performance depends heavily on proper bandwidth selection.

Introduction

1. Restatement of the problem

Given a set of two-dimensional data: Data4Regression, where Sheet1 contains the training data and Sheet2 contains the test data.

1) Perform linear regression on the data using the least squares method, gradient descent (GD), and the Newton method, respectively, and observe the training error and test error.

2) If you find that the linear model does not fit the data well (since the data is actually nonlinear, it is normal for the experimental results in the previous question to be poor), is it possible to find a more suitable model to fit the given data? Please provide the reasons for your choice of model, the specific experimental results, and an analysis of those results.

2. Data Analysis

The data sets are visualized as shown below, displaying the training set, the test set, and a combined view of both sets in a single graph. As can be seen from the graph, the training and test sets exhibit poor linearity; therefore, using linear fitting alone may not yield satisfactory results.

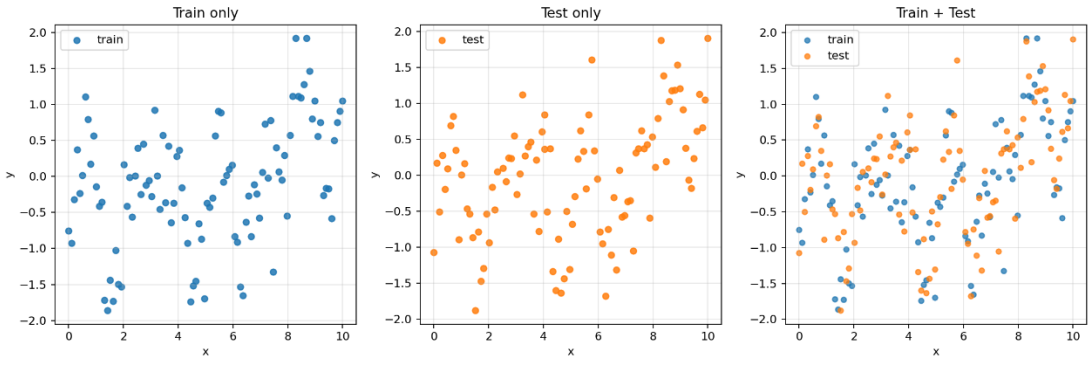

Figure 1: Visualization of Raw Data

Based on our analysis of the data, in the following sections we will introduce the linear and nonlinear regression models used in our experiments, present the results of applying these models to the given data, and analyze those results. We will conclude with a summary of the entire paper in the final section.

3. Error evaluation metrics

To assess the performance of different models and algorithms, we employ two widely used metrics: Root Mean Squared Error (RMSE) and the coefficient of determination (R^2). The RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

where $y_i$represents the true value and $\hat{y}_i$ represents the predicted value. RMSE measures the average magnitude of the prediction errors, with a lower value indicating that the model predictions are closer to the actual values. Since RMSE is expressed in the same units as the target variable, it provides an intuitive interpretation of the model's predictive accuracy.

The coefficient of determination, denoted as R^2, is given by:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the true values. The R^2 value ranges from 0 to 1, with values closer to 1 indicating that the model explains a greater proportion of the variance in the dependent variable.

Methodology

1.M1: Linear Regression with Least Squares

The least squares method provides a closed-form solution for linear regression by minimizing the sum of squared residuals. Given the training data $\{(x_i, y_i)\}_{i=1}^{n}$, the model is

defined as $y = w_0 + w_1 x$. The parameters $\mathbf{w} = [w_0, w_1]^T$ are obtained by solving the normal equation:

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$$

where X is the design matrix augmented with a column of ones, and $\mathbf{y}$ is the vector of target values. This method yields the optimal parameters in a single step without requiring iterative optimization.

2.M2: Linear Regression with Gradient Descent

Gradient descent is an iterative optimization algorithm that minimizes the cost function by updating parameters in the direction of the negative gradient. Given the training data $\{(x_i, y_i)\}_{i=1}^n$ and the linear model $\hat{y}_i = w_0 + w_1 x_i$, the cost function is defined as the mean squared error (MSE):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

where $\mathbf{w} = [w_0, w_1]^T$. The gradient of the cost function is:

$$\nabla J(\mathbf{w}) = -\frac{2}{n} X^T (\mathbf{y} - X\mathbf{w})$$

where X is the design matrix augmented with a column of ones, and $\mathbf{y}$ is the vector of target values. The parameters are updated iteratively using the rule:

$$\mathbf{w} := \mathbf{w} - \alpha \nabla J(\mathbf{w})$$

where α is the learning rate. In this experiment, the learning rate is set to 1×10^{-3}, and the algorithm runs for 20,000 iterations to ensure convergence.

3.M3: Linear Regression with Newton's Method

Newton's method is a second-order optimization algorithm that uses both the gradient and the Hessian matrix to update parameters. Given the training data $\{(x_i, y_i)\}_{i=1}^n$ and the linear model $\hat{y}_i = w_0 + w_1 x_i$, the cost function is the mean squared error (MSE):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

where $\mathbf{w} = [w_0, w_1]^T$. The gradient $\nabla J(\mathbf{w})$ is defined as in gradient descent, and the Hessian matrix is:

$$H = \frac{2}{n} X^T X$$

where X is the design matrix augmented with a column of ones. The update rule for Newton's method is:

$$\mathbf{w} := \mathbf{w} - H^{-1} \nabla J(\mathbf{w})$$

For linear regression, the cost function is convex and quadratic, allowing Newton's method to converge in a single iteration.

4.M4: Polynomial Regression

To capture nonlinear patterns in the data, we extend the linear model by introducing higher-order powers of the input variable. For training samples $\{(x_i, y_i)\}_{i=1}^n$, the polynomial regression model with degree $d$takes the form:

$$\hat{y}_i = \theta_0 + \theta_1 x_i + \theta_2 x_i^2 + \cdots + \theta_d x_i^d$$

The coefficients $\theta_0, \theta_1, \ldots, \theta_d$ are unknown parameters to be determined. By incorporating these polynomial terms, the model gains the ability to represent curved relationships that a straight line cannot capture. Parameter estimation is performed by minimizing the sum of squared differences between observed and predicted values:

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

This optimization problem admits a closed-form solution through the normal equation:

$$\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$$

where the design matrix X contains the polynomial features $[1, x_i, x_i^2, \ldots, x_i^d]$ for each observation, and $\mathbf{y}$ collects the corresponding target values. One practical concern with this approach is that choosing a very high degree may cause the model to fit noise rather than the underlying signal, a phenomenon known as overfitting. Hence, selecting an appropriate polynomial degree is crucial for achieving good generalization performance.

5.M5: Kernel Regression

Instead of assuming a fixed parametric form, kernel regression adopts a non-parametric approach that estimates the output based on local similarity to training examples. For a training set $\{(x_i, y_i)\}_{i=1}^n$, the prediction at a new point x is formed as a weighted combination of the observed responses:

$$\hat{y}(x) = \frac{\sum_{i=1}^{n} K(x, x_i) y_i}{\sum_{i=1}^{n} K(x, x_i)}$$

The weight assigned to each training sample is determined by the kernel function $K(x, x_i)$, which measures how close x is to x_i. The shape of the weighting function is controlled by a bandwidth parameter that adjusts the smoothness of the estimate. In our experiments, we adopt the Gaussian kernel, which is widely used for its smooth and localized properties:

$$K(x, x') = \exp\left(-\gamma \parallel x - x' \parallel^2\right)$$

Here, γ regulates the effective range of influence each training point exerts. A smaller γproduces a smoother function that averages over a broader neighborhood, while a larger γ makes the estimate more sensitive to nearby points. The performance of kernel regression hinges on proper selection of both the kernel type and the bandwidth, as these directly affect the bias-variance trade-off.

Experimental Studies

3.1 Linear Regression with Different Optimizers

We begin by examining the performance of linear regression under three distinct optimization strategies: the ordinary least squares (OLS) estimator, iterative gradient descent (GD), and Newton's second-order method. The linear hypothesis takes the form $f(x) = w_0 + w_1 x$, and we adopt mean squared error (MSE) as the loss function. For GD, the step size is fixed at $\alpha = 1 \times 10^{-3}$and the procedure runs for 20,000 steps to guarantee convergence. Newton's method exploits the Hessian information and converges immediately because the objective is quadratic.

Table 1: Comparative results for linear regression using three optimization approaches

method			*Training Set*		*Test Set*	
	w_0	w_1	*RMSE*	R^2	*RMSE*	R^2
Ordinary Least Squares	-0.648747	0.108947	0.78321	0.14127	0.77140	0.132873
Gradient Descent	-0.648715	0.108943	0.78321	0.14127	0.77140	0.132875
Newton's Method	-0.648747	0.108947	0.78321	0.14127	0.77140	0.132873

The outcomes presented in Table 1 show that all three optimization methods reach essentially the same solution, which is unsurprising given they minimize the same convex loss function. Minor numerical discrepancies between GD and the other two methods fall

within the range of floating-point precision and do not indicate any meaningful difference in solution quality. In terms of computational cost, Newton's method stands out by requiring only one iteration, while GD demands tens of thousands of updates. The closed-form OLS solution offers a direct computation without iterative tuning.

Examining the error metrics reveals consistently high RMSE values (approximately 0.78 on training and 0.77 on testing) alongside low R^2 scores (around 0.14). These figures strongly suggest that a linear model cannot adequately represent the relationship between the input and output variables, implying the presence of nonlinearity in the data.

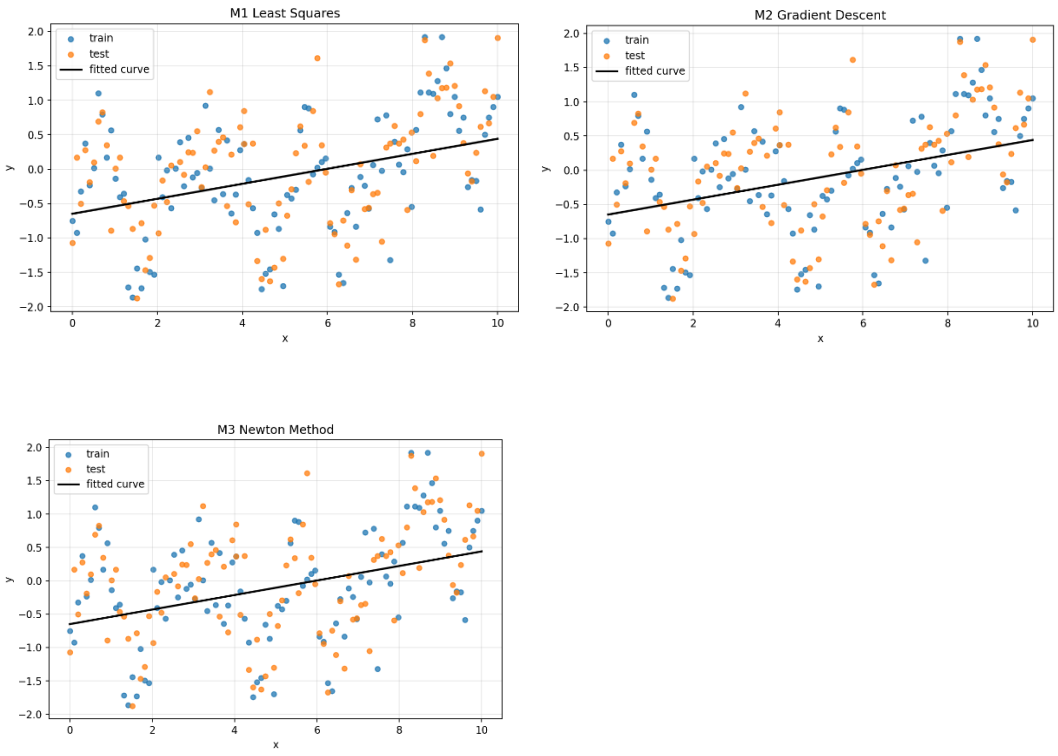

Figure 2: Visualization of linear regression fits

The visualization in Figure 2 further confirms this observation. The fitted straight line fails to track the curvature evident in the data distribution, systematically deviating from the actual values across different input ranges. This visual evidence aligns with the quantitative results, reinforcing the need for more expressive models capable of capturing nonlinear patterns.

3.2 Polynomial Regression

We next investigate whether increasing model flexibility can improve predictive performance. Polynomial regression allows the model to capture curvature by including higher-order powers of the input variable. We evaluate polynomial models with degrees ranging from 1 to 20 and report the corresponding training and test errors in Table 2. All models are estimated using ordinary least squares without regularization to examine the raw fitting capability of polynomial bases.

Table 2: Performance of polynomial regression models across different polynomial degrees

Degree	*Training Set*		*Test Set*	
	RMSE	R^2	*RMSE*	R^2
1	0.7832	0.1413	0.7714	0.1329
2	0.7519	0.2084	0.7312	0.2209
3	0.7519	0.2086	0.7327	0.2178
4	0.7456	0.2218	0.7361	0.2104
5	0.7247	0.2648	0.7177	0.2493
6	0.7247	0.2648	0.7177	0.2494
7	0.6820	0.3489	0.6805	0.3252
8	0.6793	0.3541	0.6797	0.3268
9	0.5951	0.5043	0.6225	0.4354
10	0.5914	0.5104	0.6195	0.4408
11	0.5495	0.5773	0.5784	0.5125
12	0.5581	0.5639	0.5884	0.4956
13	0.6179	0.4655	0.6270	0.4271
14	0.6279	0.4481	0.6360	0.4106
15	0.6902	0.3330	0.6909	0.3044
16	0.6925	0.3286	0.6897	0.3068
17	0.6925	0.3286	0.6897	0.3069
18	0.6935	0.3268	0.6897	0.3068
19	0.6962	0.3215	0.6921	0.3020
20	0.6999	0.3142	0.6958	0.2944

From the results, we observe a clear trend: as the polynomial degree increases from 1 to 11, both training and test errors decrease substantially, accompanied by rising R^2 values. The best test performance occurs at degree 11, achieving a test RMSE of 0.5784 and an R^2 of 0.5125, which represents a significant improvement over the linear baseline (degree 1). This confirms that the underlying relationship between $x$and $y$is nonlinear, and higher-degree polynomials can better approximate this structure.

However, beyond degree 11, performance begins to deteriorate. Test RMSE rises again, and R^2 drops, indicating that the model starts to overfit the training data. At very high degrees (e.g., 13 and above), the fitted polynomial becomes excessively wiggly, capturing noise rather than the true signal. This behavior is typical in polynomial regression: increasing complexity too much leads to poor generalization.

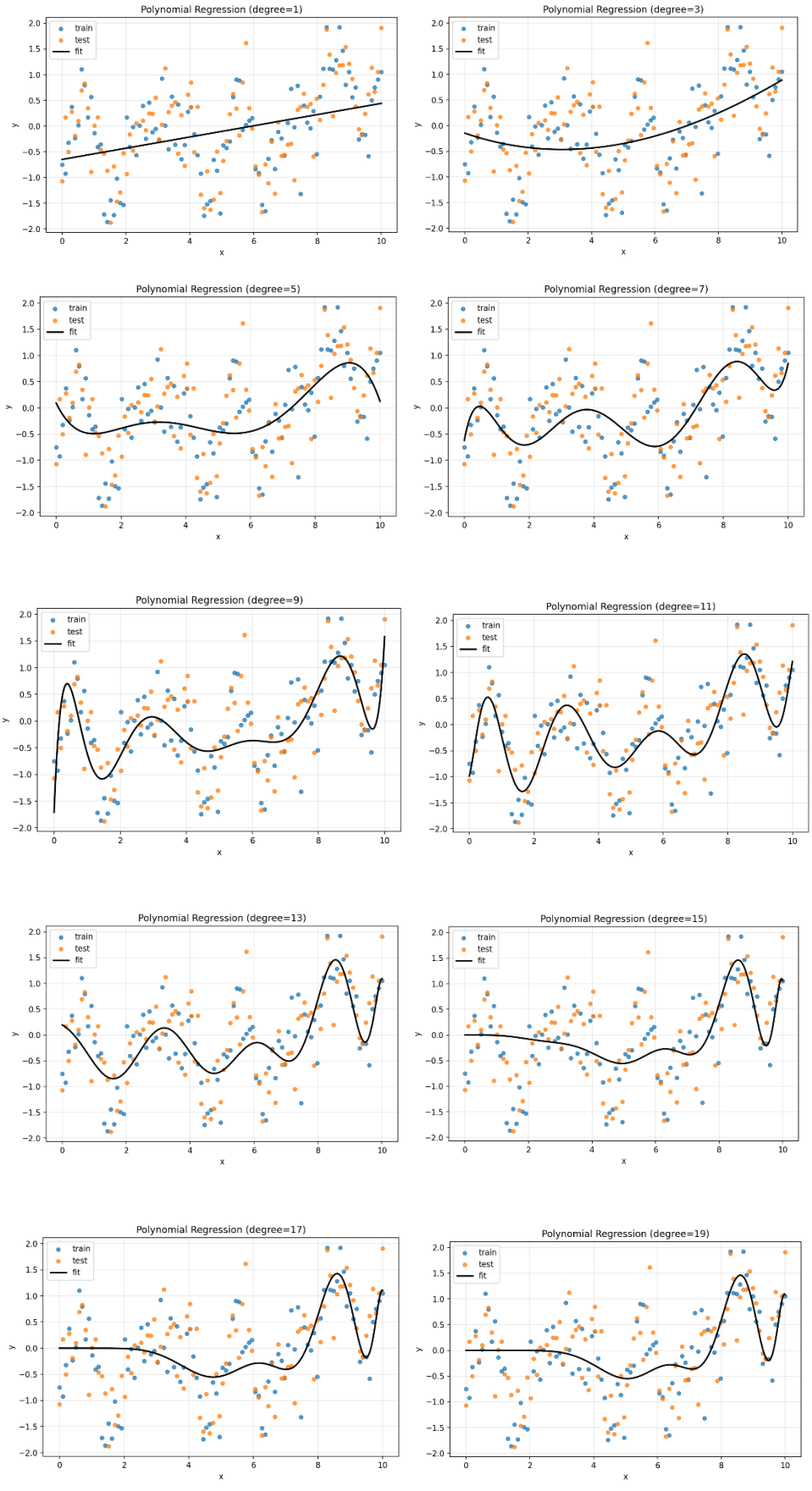

Figure 3: Polynomial regression fits for selected degrees

The visualizations in Figure 3 further illustrate this phenomenon. Low-degree polynomials (e.g., degree 1 to 5) produce overly smooth curves that cannot follow the data distribution. As the degree increases to 11, the fitted curve aligns more closely with the observed points, capturing the curvature appropriately. Beyond that, higher-degree polynomials (13, 15, 17, 19) exhibit excessive oscillations, particularly near the boundaries, indicating overfitting. Among all tested degrees, the 11th-degree polynomial achieves the best balance between fit and generalization.

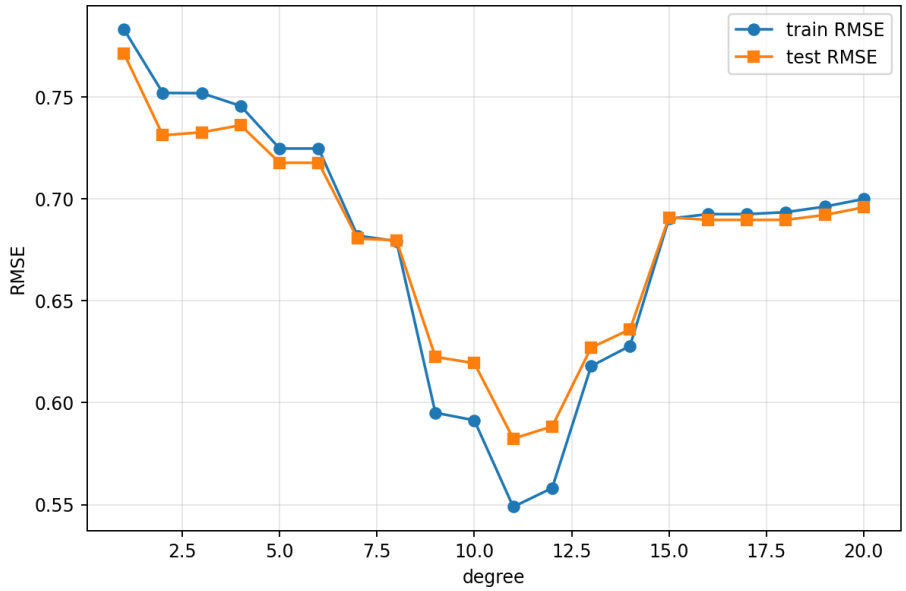

Figure 4: Graph of RMSE as a function of degree

3.3 Kernel Regression with Gaussian Kernel

As an alternative to parametric polynomial models, we explore kernel regression, a non-parametric approach that estimates the output as a weighted average of nearby training points. The Gaussian (RBF) kernel is employed, which assigns higher weights to points closer to the query location. The bandwidth parameter γ(or equivalently, the kernel width) controls the smoothness of the resulting function. We test a range of bandwidth values from 0.05 to 2.0, and the corresponding performance metrics are summarized in Table 3.

Table 3: Performance of Gaussian kernel regression under different bandwidth settings

Bandwidth	*Training Set*		*Test Set*	
	RMSE	R^2	*RMSE*	R^2
0.05	0.1152	0.9814	0.5673	0.5310
0.10	0.3178	0.8586	0.5007	0.6347
0.15	0.3791	0.7988	0.4925	0.6465
0.20	0.4171	0.7564	0.4984	0.6380
0.25	0.4499	0.7167	0.5109	0.6197
0.35	0.5151	0.6285	0.5466	0.5646
0.50	0.6004	0.4953	0.6048	0.4670
0.75	0.6804	0.3520	0.6697	0.3464
1.00	0.7165	0.2814	0.7031	0.2795
1.25	0.7343	0.2451	0.7209	0.2426
1.50	0.7451	0.2227	0.7324	0.2183
2.00	0.7607	0.1899	0.7497	0.1809

The results reveal a strong dependence on the bandwidth parameter. When the bandwidth is very small (e.g., 0.05), the model becomes highly localized, achieving an extremely low training RMSE of 0.1152 and an R^2 of 0.9814, indicating near-perfect interpolation of the training data. However, the test performance is considerably worse (RMSE = 0.5673), suggesting severe overfitting. As the bandwidth increases to around 0.15–0.20, the model smooths out and generalization improves. The best test performance is observed at a bandwidth of 0.15, with test RMSE = 0.4925 and test $R^2 = 0.6465$, outperforming the best polynomial model (degree 11) by a noticeable margin.

As the bandwidth continues to grow beyond 0.5, the model becomes increasingly smooth and loses the ability to capture local variations. At bandwidth 2.0, both training and test errors rise significantly, and R^2 drops below 0.2, indicating underfitting. This trade-off is typical in kernel-based methods: a small bandwidth leads to high variance and overfitting, while a large bandwidth introduces high bias and underfitting.

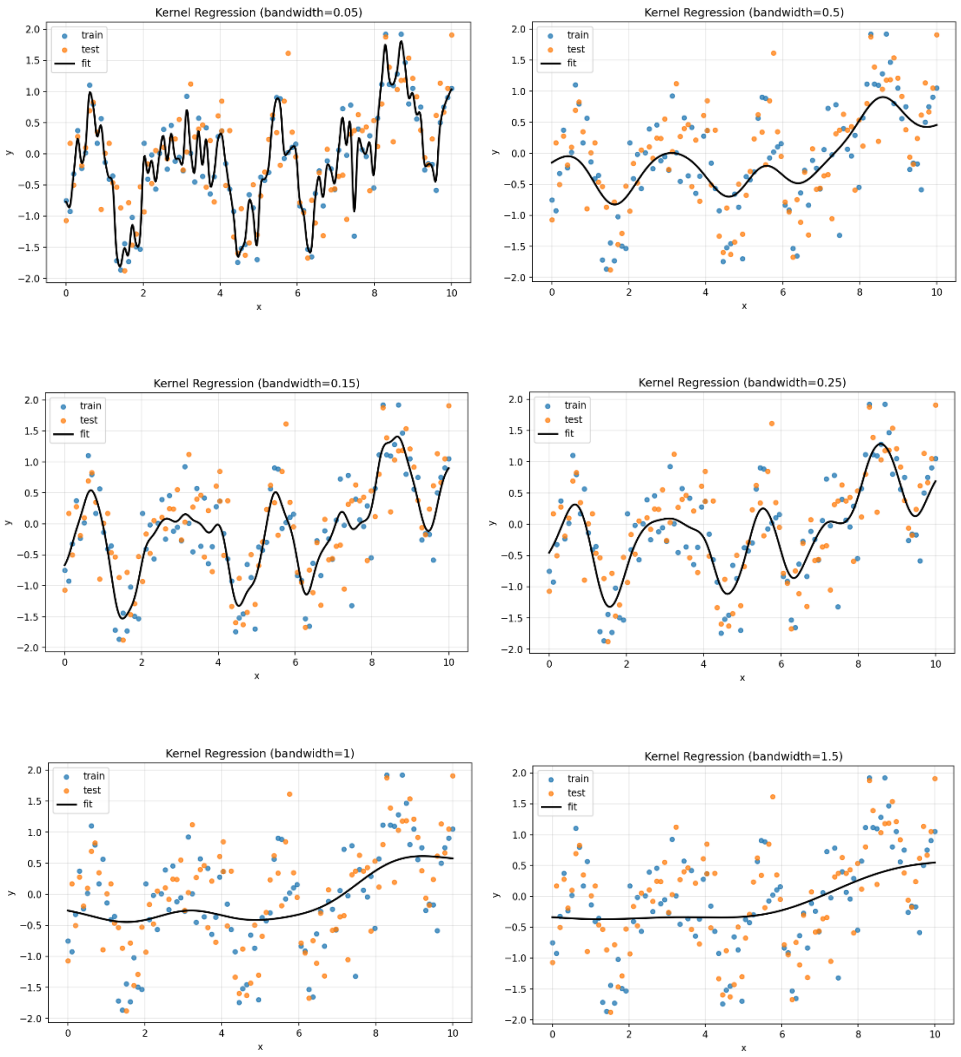

Figure 5: Kernel regression fits for different bandwidth values

The visualizations in Figure 5 illustrate how bandwidth affects the fitted function. With a very small bandwidth (0.05), the estimate passes through almost every training point, resulting in a highly wiggly curve that fails to generalize. As bandwidth increases to 0.15, the curve becomes smoother while still tracking the overall data trend, achieving the best test performance. Further increasing the bandwidth to 0.5, 1.0, and 1.5 produces increasingly flatter estimates that miss important curvature, eventually approximating a nearly linear fit at the largest bandwidth values.

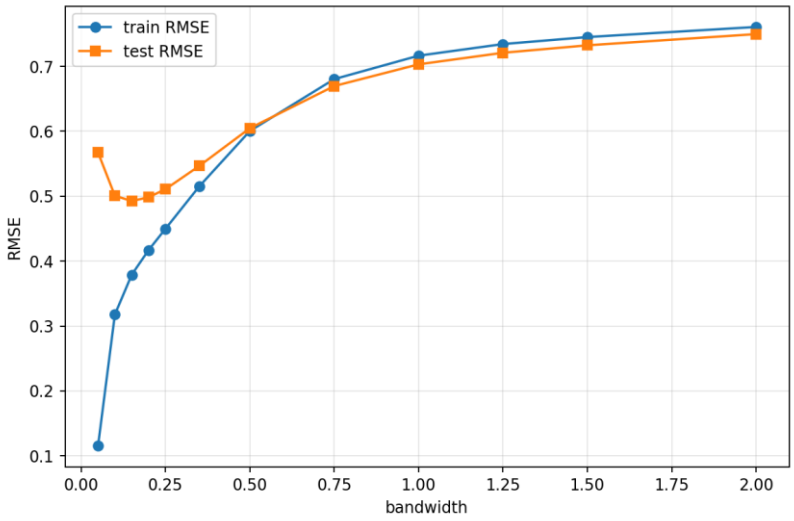

Figure 6: Graph of RMSE as a function of bandwidth

Overall, kernel regression with an appropriately chosen bandwidth (around 0.15) achieves the best generalization among all models tested, outperforming both linear and polynomial regression in terms of test RMSE and R^2.

Conclusions

In this report, we applied linear regression using three optimization methods—least squares, gradient descent, and Newton's method—and found that all produced nearly identical results. The linear model performed poorly, with test RMSE of 0.771 and R^2 of 0.133, confirming that the data is inherently nonlinear.

To address this limitation, we explored polynomial regression and kernel regression. Polynomial regression achieved its best performance at degree 11, with test RMSE of 0.578 and R^2 of 0.513. Higher degrees led to overfitting, as indicated by deteriorating test performance and excessive curve oscillations.

Kernel regression with the Gaussian kernel outperformed both linear and polynomial models. With an optimal bandwidth of 0.15, it achieved test RMSE of 0.493 and R^2 of 0.647, representing the best generalization among all methods tested. Smaller bandwidths caused severe overfitting, while larger bandwidths resulted in underfitting.

Overall, our results demonstrate that nonlinear models are essential for capturing the underlying structure of this dataset. Kernel regression, with proper bandwidth selection, proved to be the most effective approach, highlighting the importance of balancing model complexity to achieve good generalization.

References

[1] Bishop C M. Pattern Recognition and Machine Learning[M]. New York: Springer, 2006.